

Debugger

⇒ Set a breakpoint
(Execution stops here; and we check what are the different values of the variables)

⇒ The actual tool used is called gdb.

gcc -g abc.cpf
gdb a.out

⇒ There is an extra package with the suffix debuginfo.

In case of segmentation fault, can you figure out the reason?

A file called "core.out" is created which stores the stack content. By giving the command "backtrace", you can find the values assigned to each variable.

valgrind $\Rightarrow$ valgrind ./a.out

C and C++ do not run in managed environment, but Java and Python do. So, a seg fault in C/C++ is an exception to the processor.

⑩ Portable $\Rightarrow$ Managed environments
are more portable

⑩ Security $\Rightarrow$ Better protection of
the HW is possible.

⑩ Easier debugging $\Rightarrow$ Since it is
farther from the HW, so it is
difficult to introduce bugs related to
memory.

Dynamic analysis of code; which
means that not all bugs can be
discovered. To install software in
critical systems, some form of safety
analysis is required.

$\leadsto$ valgrind --tool=cachegrind ./a.out

L1D cache miss

L1I cache miss

L2 cache miss

L3 cache miss

Warnings $\Rightarrow$ Bad programming practices or incorrect logic.

gcc abc.cpp -Wall

Always switch on this flag to ensure your code uses proper logic.

```
int *p;
```

```
p = (int *) malloc(sizeof(int) * 10);
```

Scope of a variable

Block of code within which the variable name is valid.

```

int main(void) {
    int i = 10;
    for (int i = 0; i <= 15; i++) {
        cout << i;
    }
    cout << i;
}

```

Two distinct 'i' variables, each with different scope

These variables all have local scope.

But there are variables with global scope and file scope.

```

abc.cpp
static int gvar = 10;
int main(void) {
    ;
}

```

```

def.cpp
static extern int gvar;
int fun() {
    gvar = 5;
    ;
}

```

gcc -g q1.cpp -O q1-debug.out

gcc -O2 q1.cpp -O q2-opt.out